



CS344: Operating Systems Lab

Lab Quiz, October 23, 2024

Set - B

Indian Institute of Technology Guwahati

Duration: 40 Minutes

Total : 20 Marks

Name: _____

Roll No.: _____

Student Signature: _____

Invigilator Signature: _____

(10 Questions)

PART-A

[10 Marks]

> Please mention only the option you select inside the box in capital letters.

Q1. Which of the following best describes the role of wait2() of the assignment?

- a>** It tracks the process execution states but does not return any time information to the user program.
- b>** It only tracks the total runtime of the process.
- c>** It waits for all child processes to finish and then returns the total sleep time of the system.
- d>** It calculates the cumulative time spent by a process in the SLEEPING, RUNNABLE, and RUNNING states.

Q2. What happens when the kernel tries to allocate memory for a process and there is no free physical memory available in the xv6 swapping mechanism?

- a>** The kernel swaps out pages to free memory and suspends the
- b>** The process is immediately terminated by the kernel.

requesting process.

c> The kernel reduces the process's memory allocation to fit available memory.

d> The process enters a waiting state until other processes release memory.

Q3. What is the role of `usys.s` in `xv6`?

a> It contains the assembly code that transitions between user space and kernel space for system calls.

b> It contains the user-level implementation of system calls.

c> It provides the definitions for system call interfaces in the kernel.

d> It holds the process control block (PCB) structure for each process.

Q4. How does the x86 architecture handle the translation of large memory spaces (above 4 GB) using its page table structure, and what specific limitations does this impose?

a> By using an 8-bit page table, which limits the memory space to 256 MB.

b> By using a two-level page table system where the top 10 bits of the virtual address select the page directory, limiting the accessible memory.

c> By using a 16-bit page directory, which extends the physical memory limit beyond 8 GB.

d> By using a two-level page table system where the last 10 bits of the virtual address select the page directory, limiting the accessible memory.

Q5. In order to safely access and modify the process table (`ptable`), which of the following should be done before reading or writing to it in a system call like `getprocessesinfo()`?

- a> Interrupt the running process.
- b> Set process priority to avoid interruptions.
- c> Acquire ptable.lock before accessing the table.
- d> Disable interrupts.

Q6. When implementing the user program to display animation in, what file should you modify to ensure the binary is automatically included in the filesystem of xv6 upon running make qemu?

- a> user/sleep.c.
- b> user/echo.c.
- c> User/ulib.c.
- d> Makefile.

Q7. What does the p->pgdir field in the struct proc represent in xv6?

- a> The process's execution state (e.g., running, waiting, exiting).
- b> The process's page table, used for address translation.
- c> The pointer to the process's user stack.
- d> The priority level of the process.

Q8. When implementing lazy memory allocation in xv6, how can the kernel identify the virtual address that caused a page fault, and how should it handle the faulting address during the allocation process?

- a> By using tf->trapno to check for a T_PGFLT fault and using PGROUNDDOWN(va) to round the virtual address down to the start of the page.
- b> By checking the tf->trapno for a value of T_PGFLT and directly allocating memory at the next available page boundary.

c> By inspecting the `proc->sz` field and incrementing the address by the size of the page table entry.

d> By checking if the faulting address is within kernel space and using `allocvm()` to allocate memory at the exact virtual address.



Q9. In the xv6 swapping mechanism, how can the kernel handle the potential race condition that occurs when a page fault handler tries to swap in a page that is already being swapped out by another kernel process?

a> By using a blocking semaphore to suspend the page fault handler until the swapping-out process finishes.

b> By employing a busy-wait loop in the page fault handler to continuously check the status of the page until it is swapped back in.

c> By terminating the page fault handler and rescheduling the process when the page is already in the process of being swapped out.

d> By introducing a reference count for each page, allowing the kernel to wait for the page to become available if it's currently being swapped out.



Q10. Consider a hybrid [Shortest Job First (SJF) + Round Robin] scheduling algorithm and four processes P1, P2, P3, and P4. The processes come to the Ready Queue at 0- time interval. The burst time for the processes P1, P2, P3, P4 are 10, 12, 8 and 5 respectively.

The quantum of time for scheduling is fixed as the burst time of the primary process in the Ready Queue. What is the order in which the processes are set up in the Ready Queue and the time quantum?

a> P4, P3, P1, P2 and 5-time units.

- b>** P1, P2, P3, P5 and 8-time units.
- c>** P2, P3, P1, P4 and 5-time units.
- d>** P2, P1, P3, P4 and 12-time units.

PART-B

Q1. When the PC is first powered on, at which physical address does the processor start executing instructions?

- a>** 0x00000000.
- b>** 0x000A0000.
- c>** 0xF0000000.
- d>** 0x000FFFF0.

Q2. Which file in xv6 contains the array of function pointers for system calls?

- a>** trap.c.
- b>** syscall.c.
- c>** bootmain.c.
- d>** sysproc.c.

Q3. In real mode (the mode that PC starts off in, before xv6 OS is booted), how are addresses calculated from CS (segment) and IP (offset) registers ?

- a>** segment + (offset << 4).
- b>** (segment x offset) + 100.
- c>** (16 x segment) + offset.
- d>** (segment | 0xFF00) + (offset & 0xF0A0).

Q4 You have implemented a lottery scheduler in xv6, and you observe that a process with 30 tickets is scheduled for 20 ticks over 100 scheduling events. Assuming no bias in randomness, what could be the issue with your implementation?

- a>** The total number of tickets in the system is too low.
- b>** The process is not being added to the scheduling queue properly.
- c>** The system is too slow to generate enough ticks.
- d>** The random number generator has a small range.

Q5. If the time complexity of worst case Round Robin Scheduling Algorithm is $O(n^2)$, what is the best case time complexity of Shortest Job First?

- a>** $O(1)$.
- b>** $O(n \log n)$.
- c>** $O(n)$.
- d>** $O(\log n)$.

Q6. In which scenario will a process be marked as SLEEPING?

- a>** When it is waiting for I/O.
- b>** When it is running.
- c>** When it is ready to run.
- d>** When it has terminated.

Q7. In the modified sbrk() function, what is tricked by incrementing `proc->sz` without actual allocation?

- a>** The page table.
- b>** The CPU scheduler.

- c> The trap handler.
- d> The process into thinking it has the requested memory.

Q8. Which of the following statements is true about context switching in xv6?

- a> Context switching only happens when a process finishes execution.
- b> Context switching is performed by user-level threads.
- c> Context switching saves the state of the current process and loads the state of another process.
- d> Context switching does not involve saving the process state.

Q9. Which system call returns information about processes, including their pid, ticks, and tickets?

- a> getinfo().
- b> getprocessesinfo().
- c> processinfo().
- d> ps().

Q10. In the context of lazy memory allocation implemented in xv6, what is the main reason for receiving a trap 14 (page fault) when \ running simple commands like echo hi?

- a> The sbrk() system call allocates more memory than required.
- b> The memory requested by sbrk() is not allocated until it is accessed.
- c> Physical memory gets exhausted when the command is run.
- d> The process exceeds its virtual memory limit.